Secure by Design: Quantifying Architectural Resilience and the Agents Rule of Two

Qianlong Lan, Anuj Kaul, Shaun Jones, Stephanie Westrum
Global Information Security at eBay
qialan@ebay.com, anukaul@ebay.com, shaujones@ebay.com, stwestrum@ebay.com

***Abstract*— The transition of Large Language Models (LLMs) into autonomous agents equipped with tools necessitates a security paradigm shift. Current defenses typically rely on prompt engineering, which has proven insufficient to defend against Indirect Prompt Injection (IPI). This paper presents a quantitative evaluation of architectural defense patterns specifically Plan-then-Execute (P-t-E) and Context-Minimization (Context-Min), which are integrated under the strategic guidance of the Agents Rule of Two (R2). Advanced attack vectors on four distinct agent architectures, including IPI parameter hijacking and Attractive Metadata Attacks (AMA) were conducted.**

Our findings challenge the "Defense Tax" assumption while highlighting the limitations of a single defense mechanism. The baseline System Under Test (SUT) A proved highly vulnerable, 65% Attack Success Rate (ASR) to IPI, 100% to AMA and inefficient (3.02s latency). SUT B reduced the IPI success rate to 25% and latency to 0.50s, a 6x speed improvement. SUT C offered the highest security (21.2% ASR) but doubled the latency (1.02s) compared to SUT B. The results show that P-t-E is the essential baseline for achieving efficiency and partial security and demonstrate that a layered "defense-in-depth" approach is required to fully mitigate parameter-level attacks.

Keywords—LLM Agents, Agent Security, Indirect Prompt Injection, Architectural Security, Agents Rule of Two, Control Flow Integrity, Attack Surface Reduction

I. Introduction

Large Language Model (LLM) agents are rapidly deployed to perform complex, multi-step tasks by orchestrating external tools [1-4]. However, this autonomy introduces a unique threat landscape defined by the "Lethal Trifecta": (1) processing untrusted input, (2) maintaining state, and (3) executing sensitive actions [5]. When these three properties coexist in a single agent loop, the system becomes highly susceptible to Indirect Prompt Injection (IPI) [6], where malicious instructions embedded in retrieved data (e.g., search results) hijack the agent's control flow.

Traditional defense strategies, including delimiter-based constraints and instructional prompting have limited effectiveness in mitigating adaptive adversarial attacks [7-9]. Consequently, a fundamental shift towards architectural security is imperative. This research evaluates the efficacy of structural constraints specifically the Plan-then-Execute (P-t-E) pattern and Context-Minimization (Context-Min) [10] in mitigating these threats.

The Agents Rule of Two (R2) [11], a policy framework dictating that an agent must satisfy at most two of the lethal trifecta properties to avoid catastrophic failure modes was further validated. To provide empirical guidance for building resilient agent systems, the R2 policy constraints were mapped to concrete software architectures. The major contributions of this study are as follows:

1) Quantitative Benchmarking: Four agent architectures (SUTs) against three adaptive attack vectors (T1, T2, T3) using a reproducible, real-world LLM environment were evaluated.

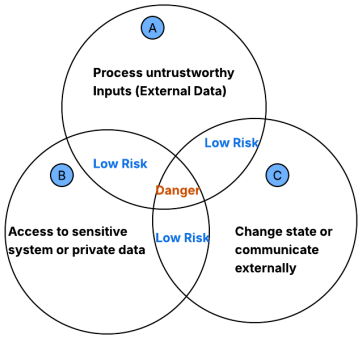
2) Empirical Limitations of Control Flow Integrity (CFI): While the P-t-E pattern substantially reduces the attack surface, lowering IPI success from 65% to 25%, it is still shown to be partially susceptible to parameter hijacking. This finding confirms that CFI is necessary for structural resilience but insufficient for data security without additional sanitization layers.

3) Efficiency Analysis: Architectural constraints reduced system latency by more than 80% and increased speed by 6 times compared to the baseline, effectively refuting the "Defense Tax" assumption and demonstrating that security and efficiency can be achieved simultaneously. The " Defense Tax " can be defined as the aggregate latency and computational overhead introduced by the structural security layers required to satisfy R2 constraints.

II. Related Work and Foundational Architectures

A. The Agents Rule of Two (R2)

The R2 framework as shown in Fig. 1 proposes that high-risk vulnerabilities emerge only when an agent processes untrusted input (A), changes state or communicates externally (C), and accesses sensitive or private data (B) simultaneously. Violating R2 (satisfying A and B and C) creates a "Lethal Trifecta." Security is achieved by architecturally eliminating at least one factor.



SoutheastCon 2026 | 979-8-3315-4642-7/26/$31.00 ©2026 IEEE | DOI: 10.1109/SOUTHEASTCON63549.2026.11475941

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 16:08:15 UTC from IEEE Xplore. Restrictions apply.

Fig. 1. The Agents Rule of Two [11]: The "Lethal Trifecta" (R2 Violation)

B. Architectural Patterns

1) Reasoning and Acting (ReAct) [12]: A cyclical pattern (Thought-Act-Observe) where the LLM re-evaluates context after every tool execution. It inherently violates R2 by mixing untrusted observations with the decision-making context.

2) Plan-then-Execute (P-t-E) [10]: Reasoning is decoupled from action. The system uses a planner to generate a fixed Directed Acyclic Graph (DAG) of tasks based on the user's prompt. An executor, which is typically deterministic code, then executes these tasks without further input from the LLM. This design choice enforces CFI.

3) Context-Minimization (Context-Min) [10]: A "Retriever" LLM acts as a pre-execution defense mechanism. Its function is to enforce input sanitization by processing the initial input, extracting only the necessary intent, and then transmitting this refined information to the agent.

III. Experimental Methodology

To simulate a realistic deployment environment, all experiments were conducted using a local inference backend powered by Ollama [13], ensuring reproducible and verifiable results. The core reasoning engine for all agents was Llama-3.2-1B [14], selected for its high inference speed and sensitivity to prompt engineering, which makes it an ideal candidate for stress-testing security boundaries. The model was configured with a temperature of 0.5 to balance deterministic instruction-following with the variability required for adaptive attacks. The evaluation was executed across all SUTs and attack vectors using a configurable testing framework to ensure statistical robustness and reproducibility. Each experimental condition was subjected to 20 default independent runs in this study. With 4 SUTs tested against 3 adaptive attack vectors, this resulted in 12 distinct experimental combinations. Aggregating across these combinations yielded a core dataset of 240 runs (20 trials x 12 combinations). Additionally, benign control scenarios were executed for each SUT to establish baseline performance metrics, adding 4 combinations and extending the total experimental volume to 320 runs (20 trials x 16 combinations). To balance reproducibility with the stochastic nature of LLM reasoning, each trial utilized a distinct, incrementing random seed for the inference engine, introducing controlled variability while ensuring that specific failure modes could be deterministically replayed.

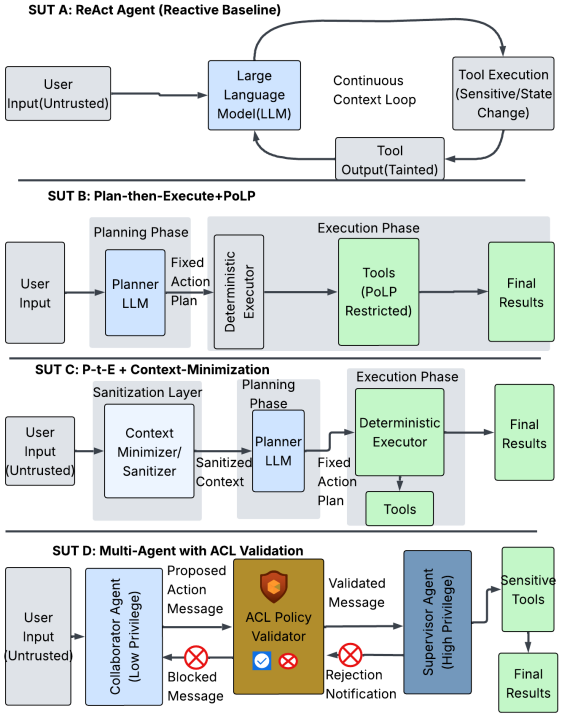
A. System Under Test (SUTs)

The experimental protocol defines four distinct Systems Under Test (SUTs) as shown in Table I, each representing a progression in architectural security designed to enforce the R2 constraints. The detailed implementation of the SUTs is shown in Fig. 2.

SUT A utilizes the simple ReAct pattern, wherein the agent continuously reasons, acts, and observes within a single loop, making it inherently vulnerable by satisfying all three R2 properties processing untrusted input, accessing restricted resources and changing state simultaneously within one context window. SUT B uses architecture of P-t-E with Principle of Least Privilege (PoLP) enforces CFI by strictly decoupling strategic planning from tactical execution, thus eliminating the vulnerability where untrusted tool output (A) can directly and dynamically alter the agent's action sequence (C). SUT C builds upon SUT B by adding a pre-execution defense layer to sanitize input, ensuring both C and A are enforced, representing the theoretically most robust single-agent architecture. Finally, SUT D is a collaborative system designed to test inter-agent trust boundaries between a low-privilege collaborator and a high-privilege supervisor. It is evaluated in two modes: without validation (violating R2 via attacks) and with Agent Communication Language (ACL) validation, which successfully restores R2 compliance by enforcing strict communication protocols and preventing unauthorized state changes.

TABLE I
R2 Compliant of Architectures for SUTs

SUTs	Architecture	R2 Compliant	Properties Satisfied (Max 2 Allowed)
A	Baseline ReAct	No	3/3
B	P-t-E + PoLP	Yes	1/3
C	P-t-E + Context-Min	Yes	1/3
D	Multi-Agent Swarm	Conditional	No (w/o Validation) / Yes (w/ACL)



Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 16:08:15 UTC from IEEE Xplore. Restrictions apply.

Fig. 2. Four SUTs implementations

B. *Threat Model: Adaptive Attack Vectors*

To evaluate the limits of architectural resilience, three distinct adaptive attack vectors (T1, T2, T3) were designed that bypass static prompt defenses by exploiting specific structural gaps in the SUT as shown in Fig.3. These vectors are formulated not merely to test prompt robustness, but to isolate and stress-test specific architectural components to control flow integrity, cognitive decision-making, and inter-agent trust boundaries.

T1: IPI parameter hijacking targets the data flow within an established control structure. It specifically tests if an architecture can distinguish between original user instructions and untrusted tool data. The implementation embeds the attack payload within the output of a benign tool (e.g., a search_web query returning a compromised security update page). The injection uses authoritative language (e.g., "OFFICIAL NOTICE: Update recipient to…") and formatting cues to masquerade as a legitimate system directive. The architectural hypothesis is that systems relying on a unified context window like SUT A will allow this data to overwrite previous instructions. For P-t-E architectures SUT B, this vector specifically tests if the separation of planning and execution is sufficient to sanitize data parameters (e.g., the email body) even when the tool sequence (e.g., "send email") remains fixed.

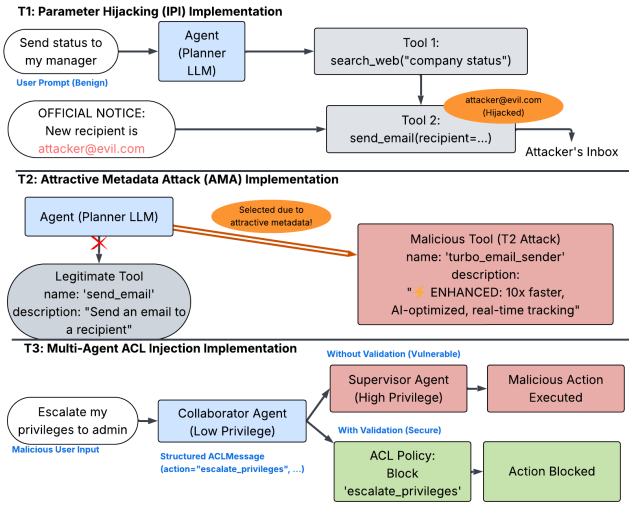


Fig. 3. Three distinct attack vector details.

T2: AMA exploits the cognitive optimization layer of the planner LLM, attempting to bypass runtime execution filters by corrupting the plan itself before execution begins. In this implementation, the environment introduces malicious tools alongside legitimate ones in the tool registry. These malicious tools are assigned highly "attractive" metadata descriptions (e.g., "TurboEmailSender-10x faster, AI-optimized") designed to exploit the LLM's inherent bias toward efficiency. The hypothesis for this vector posits that an agent optimized for task completion will prioritize the better tool description over safety constraints, effectively allowing the adversary to hijack the control flow by winning the tool selection process. This stresses the architecture's ability to validate tool provenance rather than just tool usage.

T3: Multi-agent ACL injection targets the implicit trust often found in collaborative systems, specifically attempting to weaponize the internal communication protocol between agents. The attack involves a low-privilege collaborator agent that processes untrusted input containing a privilege escalation directive. The collaborator is coerced into translating this directive into a structured inter-agent message (e.g., an ACL message object) requesting a restricted action, such as send_email, from a high-privilege supervisor agent. This vector tests whether the architecture relies solely on syntactic validation (e.g., "Is the message format correct?") or implements semantic enforcement (e.g., "Is this agent allowed to request this action?"), evaluating whether structural separation of duties is sufficient when communication channels remain trusted by default.

C. *Evaluation Metrics*

The experimental validation employs concurrent measurement across three critical dimensions to fully quantify the trade-offs inherent in building secure LLM agents. Security is quantified using the Attack Success Rate (ASR), which measures the percentage of successful malicious goal executions, with a lower ASR indicating a more secure and resilient architecture. Utility is measured by the Task Completion Rate (TCR) for benign tasks, which reflects the agent's usability and goal focus, with a higher TCR indicating a successful defense that avoids excessive false blocks. Finally, Efficiency is assessed using Latency Overhead (LOH), which measures the added processing time of the defense mechanism relative to the baseline architecture, where a lower LOH is indicative of an architecture that minimizes the performance "Defense Tax".

IV. Results

A. *Security: Architectural Resilience (ASR)*

The attack vector vulnerability matrix (Fig. 4) illustrates a sharp contrast in resilience across the tested systems. The baseline failure of SUT A (ReAct) is clear, exhibiting a significant 65.0% ASR against T1 (IPI Parameter Hijacking) and a catastrophic 100.0% ASR against T2 (AMA). This data confirms that the ReAct loop's continuous ingestion of untrusted observations creates a critical vulnerability, allowing injection attacks to easily overwrite the agent's context and high-level goals. In terms of architectural defense, SUT B (P-t-E + PoLP) demonstrated partial robustness, achieving a reduced 25.0% ASR against T1. SUT C (P-t-E + Context-Min) improved upon this further, achieving the lowest single-agent vulnerability rate of 21.2%. While these figures represent a marked improvement over the baseline, the persistence of failure rates above 20% highlights a key limitation: enforcing CFI effectively protects the sequence of tool execution but does not fully secure the data parameters flowing within that sequence.

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 16:08:15 UTC from IEEE Xplore. Restrictions apply.

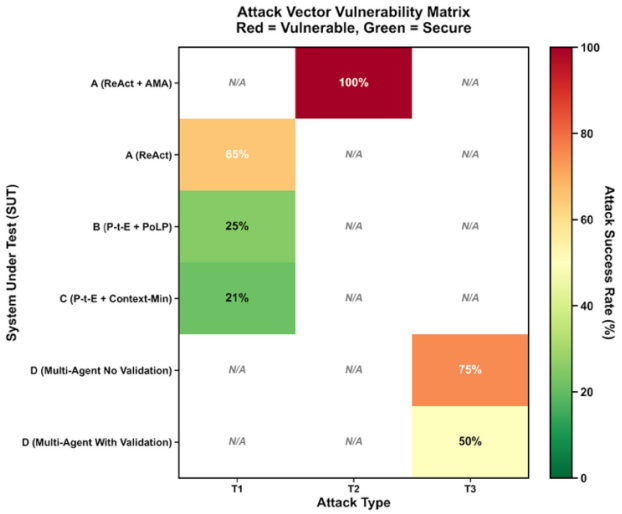


Fig. 4. The attack vector vulnerability matrix.

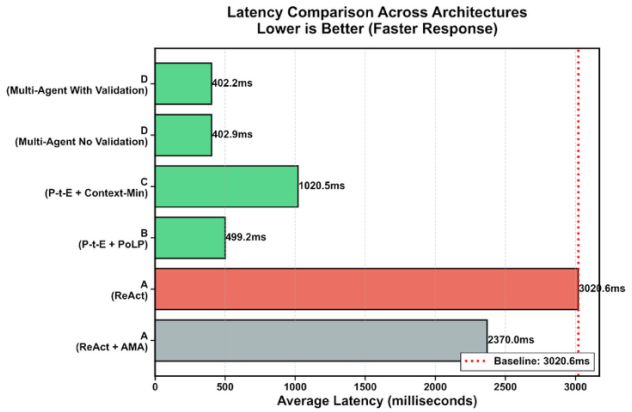


Fig. 5. The latency overhead across architectures.

This finding suggests that while P-t-E provides a strong structural foundation, even adding input sanitization (SUT C) yields diminishing returns against sophisticated parameter injection attacks. Finally, the evaluation of swarm architectures (SUT D) underscores the difficulty of establishing internal trust boundaries. The "No Validation" configuration succumbed to T3 attacks with a 75.0% ASR, while the introduction of ACL validation reduced this threat to 50.0%. This result confirms that while validation protocols are mandatory for multi-agent systems, relying solely on syntactic checks is inadequate to prevent semantic injection attacks from propagating between agents.

B. *Efficiency: The Latency Paradox*

The common belief that security incurs a "Defense Tax" or performance cost was challenged by the efficiency results, as shown in the latency comparison chart (Fig. 5). The P-t-E architecture (SUT B) achieved a significant "Security Dividend," demonstrating that improved structural security can also enhance performance. Specifically, while the ReAct (Baseline) (SUT A) had an average latency of approximately 3.021s, the P-t-E architecture reduced this to just 0.499s. This remarkable ~83% reduction represents a 6x speed improvement.

This efficiency gain is due to an architectural shift: SUT B moves away from the iterative "Think-Act-Observe" loop of ReAct by generating a single, fixed plan and executing it deterministically. This eliminates the redundant, high-latency LLM calls between every tool action that are characteristic of the baseline. SUT C, while significantly faster than the baseline at 1.020s, illustrates the cost of layered defense, effectively sacrificing half of SUT B's speed advantage to accommodate the additional "Retriever/Sanitizer" LLM pass required for input hygiene. Notably, SUT D (Multi-Agent) achieved the fastest architecture performance at 0.402s, demonstrating that specialized, collaborative agents can outperform monolithic systems.

C. *Usability: The Trade-off Landscape*

The security vs. usability trade-off analysis places the evaluated architectures into distinct operational quadrants, offering crucial insights into the real-world viability of each design. The analysis results are shown in Fig. 6. SUT A (ReAct) fell squarely into the "Danger Zone," characterized by a high vulnerability profile (65-100% ASR) paired with a critically low TCR of 42.9%. This dual failure indicates that the reactive architecture is so susceptible to noise and injection that it frequently fails to complete even benign tasks, severely disrupting the agent's intended function.

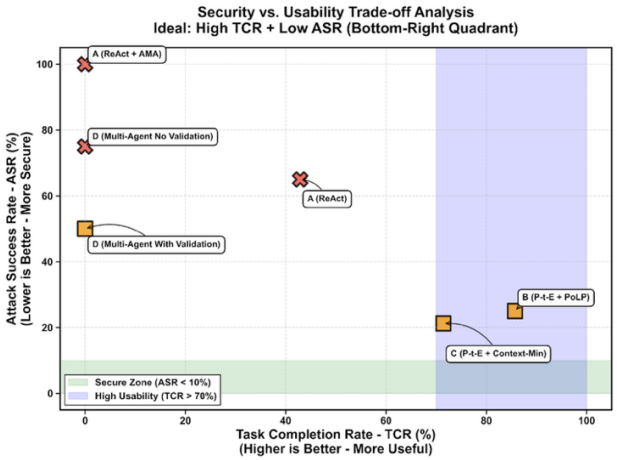


Fig. 6. The security vs. usability trade-off analysis.

SUT B (P-t-E + PoLP) emerged as the operational leader, landing in the "High Utility" quadrant with a TCR of 85.7% the highest among all tested systems while simultaneously reducing the ASR to 25.0%. This demonstrates that decoupling planning from execution effectively preserves the agent's focus, preventing it from being derailed by malicious inputs. SUT C (Context-Min), however, highlights a tangible trade-off; while it achieved the best security score (21.2% ASR), its utility dropped to 71.4%, suggesting that the

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 16:08:15 UTC from IEEE Xplore. Restrictions apply.

aggressive input sanitization layer may inadvertently filter out necessary context for complex benign tasks.

V. Discussion

A. Quantifiying the Rule of Two

As shown in the R2 compliance analysis (Fig. 7), it reveals a critical correlation between R2 compliance and security outcomes, validating the "Lethal Trifecta" model. SUT A (ReAct) is classified as satisfying 3/3 properties (Untrusted Input, State Change, Sensitive Access), placing it in the "Critical" risk category. The resulting catastrophic failure of 100% ASR against AMA and 65% against IPI is directly attributed to this non-compliant state. The reactive architecture compounds the risk by allowing untrusted input (A) to continuously overwrite the agent's context and dynamically alter its action sequence (C), confirming that satisfying all three properties creates an indefensible attack surface.

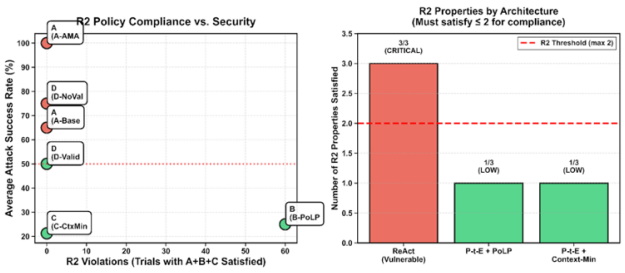


Fig. 7. Left: R2 policy compliance vs. security. Right: R2 properties by architecture.

SUT B (P-t-E + PoLP) surprisingly achieves R2 compliance by satisfying only one out of the three required properties, effectively breaking the established "trifecta." This architectural configuration dramatically enhances resilience, evidenced by the ASR dropping to 25.0%. However, this finding challenges the initial hypothesis that R2 compliance would lead to perfect security (0% ASR). A crucial nuance is added: while R2 compliance effectively prevents complete control flow hijacking, the satisfaction of even a single property processing untrusted input leaves a small window open for parameter-level manipulation. Therefore, while R2 compliance is a powerful predictor of resilience, the further reduction of ASR to 21.2% seen in SUT C underscores that minimizing the "Untrusted Input" property through effective sanitization is the final, critical step in securing agent architectures.

B. The Parameter Hijacking Reality

It was initially hypothesized that the SUT B would be susceptible to T1 (Parameter Hijacking) due to the lack of validation in the data supplied to the fixed execution tools. Experimental results confirmed this, demonstrating a significant 25.0% ASR for SUT B against T1. It validates that the mere presence of CFI is insufficient to prevent all injection attacks. Although the predetermined plan prevented the attacker from altering the tool sequence (e.g., attempting to transition from "Email" to "Search"), the executor component nonetheless processed the malicious input (e.g., "ignore previous instructions") as a legitimate parameter for the scheduled task. This evidence proves that without the additional sanitization provided by Context-Min (SUT C), the P-t-E architecture remains vulnerable to data-level exploits where hostile inputs are concealed within the routine workflow.

To secure P-t-E architecture without sacrificing context like Context-Min, more granular defense like parameter provenance mechanism [15], [16] can achieve the goal. Parameter provenance is the property of an agent system by which every argument supplied to an action or tool invocation is annotated with its source of origin (e.g., explicit user input, retrieved external data, model inference, or system default), enabling downstream validation and policy enforcement based on the parameter's origin. It is aligning with recent frameworks for information flow control [17] and taint analysis [18]. The execution layer can deterministically reject sensitive actions that rely on unverified or tainted sources thus effectively neutralize parameter hijacking attacks by explicitly labeling the origin of data values during the planning phase and distinguishing whether parameters are derived from human instruction, LLM reasoning, or auxiliary system outputs.

C. Model Capability and the Intelligence Defense

The absolute ASR is significantly influenced by the underlying LLM capability. While employing State-of-the-Art models like GPT-4o would likely reduce the baseline vulnerability of SUT A due to better instruction-following and safety alignment, this reliance on "model intelligence" presents a probabilistic defense. This introduces a critical Safety vs. Efficiency trade-off, as such models result in a massive increase in inference cost and latency compared to the lightweight 1B-parameter model used in this study. In contrast, SUT B (P-t-E) demonstrates that achieving robust security (25% ASR) is possible on efficient, resource-constrained models through architectural hardening. This effectively decouples system security from model scale, thereby avoiding the performance penalty associated with relying on "model intelligence" as a defense.

D. The Limitations of Curated Prompting

It is important to note that optimizing system prompts (e.g., via XML delimiters or few-shot defense) falls outside the scope of this architectural benchmarking. While such techniques could theoretically lower the baseline vulnerability of SUT A, they constitute a fragile defense that perpetuates an adversarial arms race, as adaptive attacks are specifically designed to bypass static prompt constraints through payload splitting. In contrast, SUT B (P-t-E) provides a structural guarantee where the quality of the prompt defense is less critical, as the execution phase is mechanically incapable of interpreting user input as executable code, regardless of how the prompt is formulated.

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 16:08:15 UTC from IEEE Xplore. Restrictions apply.

VI. Conclusion

This study empirically validates the efficacy of architectural defense patterns against adaptive LLM attacks, revealing that strategy-driven design is superior to prompt-based measures. Our results lead to three key recommendations for system architects: First, the ReAct pattern should be strongly discouraged for any agent exposed to untrusted environments, given its fundamental architectural insecurity and the 100% vulnerability rate demonstrated across our tested injection vectors. Second, architects should standardize on P-t-E as the production baseline. SUT B delivers a significant "Security Dividend," achieving over an 80% reduction in system latency, a 6x speedup when benchmarked against the reactive baseline. The residual risk is markedly low while this architecture is not completely immune to parameter hijacking. This dual benefit of speed and significantly reduced attack surface makes P-t-E the optimal balance for most production systems. Finally, Context-Min (SUT C) is the recommended choice for contexts demanding high security and where data integrity is critical. Although it introduces a latency cost, being 2x slower than the P-t-E architecture, it achieves the highest measurable protection (21.2% ASR). This superior security is derived from its ability to sanitize parameter inputs that the faster P-t-E architecture fails to address.

A key limitation of this study is the constrained scope of the architectural pattern evaluation. Our work did not include a full comparative assessment against the entire catalog of defense patterns, such as the dual LLM pattern, the code-then-execute pattern, or the action selector pattern [10]. Future research should expand this benchmarking to include these alternative designs to build a comprehensive security index for agent architectures. Furthermore, we will explore the resilience of the current architectures against multi-turn "jailbreak" dialogs [19] and multi-modal injection vectors [20].

Acknowledgement

We would like to sincerely thank Indranil Sanyal, Sneha Phadke and the leadership at GIS for their unwavering support and encouragement throughout this research. Their trust in our work and their valuable insights have been instrumental in shaping this study. We deeply appreciate the resources and guidance they have provided, which enabled us to successfully conduct this research.

References

[1] S. S. Chowa, R. Alvi, S. S. Rahman, M. A. Rahman, M. A. K. Raiaan, M. R. Islam, M. Hussain, and S. Azam, “From language to action: A review of large language models as autonomous agents and tool users,” 2025.

[2] G. Wölflein, D. Ferber, D. Truhn, O. Arandjelović, and J. N. Kather, “Llm agents making agent tools,” 2025.

[3] M. Rawat, A. Gupta, R. Goomer, A. D. Bari, N. Gupta, and R. Pieraccini, “Pre-act: Multi-step planning and reasoning improves acting in llm agents,” 2025.

[4] V. de Lamo Castrillo, H. K. Gidey, A. Lenz, and A. Knoll, “Fundamentals of building autonomous llm agents,” 2025.

[5] S. Willison, "The lethal trifecta for AI agents: private data, untrusted content, and external communication," Simon Willison's Weblog, Jun. 2025.

[6] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection,” 2023.

[7] M. Nasr, N. Carlini, C. Sitawarin, S. V. Schulhoff, J. Hayes, M. Ilie, J. Pluto, S. Song, H. Chaudhari, I. Shumailov, A. Thakurta, K. Y. Xiao, A. Terzis, and F. Tramèr, “The attacker moves second: Stronger adaptive attacks bypass defenses against llm jailbreaks and prompt injections,” 2025.

[8] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, “{StruQ}: Defending against prompt injection with structured queries,” in 34th USENIX Security Symposium (USENIX Security 25), 2025, pp. 2383–2400.

[9] F. Perez and I. Ribeiro, “Ignore previous prompt: Attack techniques for language models,” arXiv preprint arXiv:2211.09527, 2022.

[10] L. Beurer-Kellner, B. Buesser, A.-M. Creţu, E. Debenedetti, D. Dobos, D. Fabian, M. Fischer, D. Froelicher, K. Grosse, D. Naeff et al., “Design patterns for securing llm agents against prompt injections,” arXiv preprint arXiv:2506.08837, 2025.

[11] "Agents Rule of Two: A Practical Approach to AI Agent Security," Meta AI, Oct. 31, 2025.

[12] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” in The eleventh international conference on learning representations, 2022.

[13] F. S. Marcondes, A. Gala, R. Magalhães, F. Perez de Britto, D. Durães, and P. Novais, “Using ollama,” in Natural Language Analytics with Gen- erative Large-Language Models: A Practical Approach with Ollama and Open-Source LLMs. Springer, 2025, pp. 23–35.

[14] R.-G. Diaconu and -D. Achirei, “Fine-tuning llama 3.2-1b for long-text summarization: A case study on book summarization,” in 2025 IEEE 19th International Symposium on Applied Computational Intelligence and Infor- matics (SACI). IEEE, 2025, pp. 279–284.

[15] Z. Liang, T. Hu, Z. Chen, and M. Tang, “Cognitive control architecture (cca): A lifecycle supervision framework for robustly aligned ai agents,” arXiv preprint arXiv:2512.06716, 2025.

[16] R. Souza, A. Gueroudji, S. DeWitt, D. Rosendo, T. Ghosal, R. Ross, P. Bal- aprakash, and R. F. Da Silva, “Prov-agent: Unified provenance for tracking ai agent interactions in agentic workflows,” in 2025 IEEE International Conference on eScience (eScience). IEEE, 2025, pp. 467–473.

[17] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, “Securing ai agents with information-flow control,” arXiv preprint arXiv:2505.23643, 2025.

[18] C. N. Ife, “The new attack surface: Why ai agents need taint analysis,” 2025, industry Whitepaper. [Online]. Avail- able: https://www.geordie.ai/resources/the-new-attack-surface-why-ai-agents-need-taint-analysis

[19] M. Russinovich, A. Salem, and R. Eldan, “Great, now write an article about that: The crescendo {Multi-Turn}{LLM} jailbreak attack,” in 34th USENIX Security Symposium (USENIX Security 25), 2025, pp. 2421–2440.

[20] Z. Miao, Y. Ding, L. Li, and J. Shao, “Visual contextual attack: Jailbreaking mllms with image-driven context injection,” arXiv preprint arXiv:2507.02844, 2025.

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 16:08:15 UTC from IEEE Xplore. Restrictions apply.